IIMT3642: Managing and Mining Big Data - Association Rule Mining

Zhepeng (Lionel) Li, PhD
Associate Professor
Innovation and Information Management

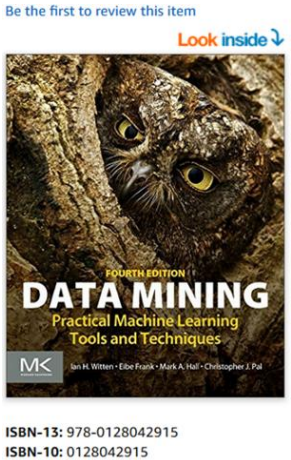
Lecture Outline

- Association Analysis
- Association Rule
  - Market Basket Model
  - Frequent Items & Association Rules
  - Apriori Algorithm
- Discussion: Mining Association Rules in Big Data Computing
  - Finding Frequent Itemsets

Association Analysis

- Association Analysis is to discover interesting relationships hidden in large item sets.
- Consider Market Basket Model for supermarket shopping scenario:
  - **Goal:** Identify items that are bought together by sufficiently many customers
  - **Approach:** Process the data that are collected with barcode scanners to find dependencies among items over sale transactions
  - **One classic rule**: if one buys diaper and milk, then s/he is also likely to grab a beer.

- What can we do if these interesting associations are discovered?
  - Cross selling
  - Merchandise placement/shelf management
  - Recommendation



- Association analysis in other areas:
  - Web: associations among Web pages visited
  - Healthcare: associations among medical tests/treatments
  - Crime/Fraud decision/Cybersecurity: associations among criminals/hackers

Association Rule

- Market-Basket Model
  - Given a large set of items, e.g., all things for sale in the supermarket
  - Given a large set of baskets, each basket contains a small subset of all items, e.g., things bought by one customer in one transaction
  - Objective is to discover association rules, e.g., customer who bought {x, y, z} tend to buy {v,w}

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Rules Discovered:

{Milk} --> {Coke}

{Diaper, Milk} --> {Beer}

- Frequent Itemsets
  - First question: how can we find sets of items that occur “frequently” together in the baskets
  - Support for itemset ***I***: number of baskets that contain all items in ***I***
    - Often measured as the fraction of the total number of baskets, i.e., a percentage
  - Given a support threshold ***s***, we call the sets of items that appear in at least ***s*** baskets frequent itemsets
  -

TID	*Items*
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Support of {Beer, Bread} = 2

- Example: Frequent Itemsets
  - Given **items** = {milk (m), coke (c), pepsi (p), beer (b), juice (j)}
  - **Support threshold** = 3 baskets
  - Observed **baskets** data are:

B1 = {m, c, b}	B2 = {m, p, j}	B3 = {m, b}
B4 = {c, j}	B5 = {m, p, b}	B6 = {m, c, b, j}
B7 = {c, b, j}	B8 = {b, c}	

  - **Frequent Itemsets** are computed as
    - 1-itemsets: {m}, {c}, {b}, {j},
    - 2-itemsets: {m,b}, {b,c}, {c,j}

- Association Rules
  - Association rules are **if-then** rules
  - $\{\boldsymbol{i_1}, \boldsymbol{i_2}, \dots, \boldsymbol{i_k}\} \rightarrow \boldsymbol{j}$ means if a focal basket contains all items in $\{\boldsymbol{i_1}, \boldsymbol{i_2}, \dots, \boldsymbol{i_k}\}$ then it is likely to have item $\boldsymbol{j}$ as well
  - So, $\{\boldsymbol{i_1}, \boldsymbol{i_2}, \dots, \boldsymbol{i_k}\}$ is called antecedent; and $\boldsymbol{j}$ is consequent
  - Nevertheless, there are often too many rules in the practice; we only focus on interesting ones. So we need to measure confidence and interests to determine rules' interestingness.
  - Confidence of a given association rule $X \rightarrow Y$ is the probability of Y given X

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

- Not all high-confidence rules are necessarily interesting.
  - For example, $X \rightarrow milk$ is of high confidence simply because milk is often bought together with any transactions, regardless of X
- Interests [optional] of an association rule $X \rightarrow Y$: difference between its confidence and the fraction of basket that contains Y

$$Interest(X \rightarrow Y) = \underline{Confidence(X \rightarrow Y)} - \underline{\Pr(Y)}$$

- Interesting rules are those with positive or negative values in this measure. Typically, we prefer positive ones that are above, say 0.5
- For simplicity, people may end up using just confidence threshold regardless the interests.

- Example: computing confidence and interest for association rule $\{m, b\} \rightarrow c$
  - Observe the same baskets data below; we define the thresholds for support and confidence are 2 and 0.5, respectively

B1 = {m, c, b}	B2 = {m, p, j}	B3 = {m, b}
B4 = {c, j}	B5 = {m, p, b}	B6 = {m, c, b, j}
B7 = {c, b, j}	B8 = {b, c}	

  - $Support(\{m, b, c\}) = 2, Support(\{m, b\}) = 4$
  - $Confidence(\{m, b\} \rightarrow c) = 2/4 = ½$
    - The rule is confident (i.e., >=0.5)
  - $\Pr(c) = 5/8$
  - $Interest(\{m, b\} \rightarrow c) = ½ - ⅝ = -1/8$
    - But it seems not interesting

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

$$Interest(X \rightarrow Y) = Confidence(X \rightarrow Y) - \Pr(Y)$$

Association Rules Mining

- **Association Rule Mining Problem:** Given a dataset of transactions, find all association rules with support ≥ ***s*** and confidence ≥ ***c***, where *s* and *c* are specified by user
  - ***Note*** *that support of a rule X→Y is the support of the union set, i.e.,*

$$Support(X \rightarrow Y) = Support(X \cup Y)$$

- **Two Steps** to address Association Rule Mining Problem
  - **Step 1: Frequent Itemsets.** Find all frequent itemsets that are above the minimum support ***s***
  - **Step 2: Rule Generation.** For each frequent itemset ***L***, find all association rules that are above the minimum confidence ***c*** in the form below:

$$Ls \rightarrow (L - Ls)$$

- **Critical Issue:** How to efficiently find the frequent itemsets in step 1
  - Big Data Challenge
    - Suppose all items $|I| = n$, the number of possible itemsets is $2^n - 1$
    - For each possible itemset, we need to scan the dataset, obtaining the actual support to determine whether it is frequent or not
    - Consequently, we need an efficient algorithm for step 1, which will find all frequent itemsets

TID	Transaction
T1	Bread, Milk, Eggs
T2	Bread, Diapers, Beer, Eggs
T3	Bread, Milk, Beer, Diapers
T4	Bread, Eggs, Milk
T5	Beer, Diapers
…	

Tips: Scanning through a large dataset normally would take longer time, especially when it is too large to fit in the memory.

- **Apriori principle:** any subset of a frequent itemset must be frequent.
  - **Example:**
    - Suppose $s = 2$ and $c = 0.6$
    - We know $support(\{Beer, Diapers\}) = 3 > 2$
    - Question: Is 1-itemset {Beer} frequent?

The answer should be "yes", since the following holds

$$Support(\{Beer\}) \geq Support(\{Beer, Diapers\})$$

TID	Transaction
T1	Bread, Milk, Eggs
T2	Bread, Diapers, Beer, Eggs
T3	Bread, Milk, Beer, Diapers
T4	Bread, Eggs, Milk
T5	Beer, Diapers

- Apriori Algorithm for step 1 (identifying all frequent itemsets of varied size)
  1. Scan data set once to find all frequent 1-itemsets
  2. Generate candidate K-itemsets from frequent (K-1)-itemsets
  3. Find all frequent K-itemsets from candidate K-itemsets by scanning dataset once
  4. Go back to #2 above
  5. The loop stops if no new candidate itemset can be generated

- Apriori Algorithm for step 1

  2. Generate candidate K-itemsets from frequent (K-1)-itemsets

     1) **Join:** Given any two frequent (K-1)-itemsets, L_1 and L_2. Take union if they satisfy the following two conditions:

        - (1) $L_1(i) == L_2(i)\ for\ i = 1, 2, \ldots, k-2$
        - (2) $L_1(k-1) <> L_2(k-1)$

- **Example:** Given frequent 2-itemsets as {1, 2}; {1, 3}; {1, 4}; {2, 3}
- How do we join for candidate 3-itemsets?

$$\{1,2\} \cup \{1,3\} = \{1,2,3\} \leftarrow$$
$$\{1,2\} \cup \{1,4\} = \{1,2,4\}$$
$$\{1,2\} \cup \{2,3\} = \{1,2,3\}\ (\text{No need to union})$$
$$\{1,3\} \cup \{1,4\} = \{1,3,4\}$$
$$\{1,3\} \cup \{2,3\} = \{1,2,3\}(\text{No need to union})$$
$$\{1,4\} \cup \{2,3\} = \ ?$$

- Apriori Algorithm for step 1

  2. Generate candidate K-itemsets from frequent (K-1)-itemsets

     1) Join

     2) Prune: remove itemsets generated in substep 2.1) if they contain any non-frequent itemset

- **Example:** Given frequent 2-itemsets as {1, 2}; {1, 3}; {1, 4}; {2, 3} and the generated pool of 3-itemsets by join are:

$$\{1,2,3\} \supset \{1,2\},\{1,3\},\{2,3\}$$
$$\cancel{\{1,2,4\}} \supset \{1,2\},\{1,4\},\boxed{\{2,4\}}$$
$$\cancel{\{1,3,4\}} \supset \{1,3\},\{1,4\},\boxed{\{3,4\}}$$

prune

Candidate 3-itemsets are:

$$\{1,2,3\}$$

In-class Exercise (Frequent Itemsets)

- **Practice**

TID	Transaction
T1	1, 3, 4, 6
T2	2, 3, 5, 7
T3	1, 2, 3, 5, 8
T4	2, 5, 9, 10
T5	1, 4

Given:
Minimum support ***s*** = 2

Try to hand compute for frequent 3-itemsets?

- **Exercise**

TID	Transaction
T1	1, 2 ,5
T2	2, 4
T3	2, 3
T4	1, 2, 4
T5	1, 3
T6	2, 3
T7	1, 3
T8	1, 2, 3, 5
T9	1, 2, 3

Given:
Minimum support ***s*** = 2

Try to hand compute for frequent 3-itemsets?

Association Rule Generation

- **Two Steps** to address Association Rule Mining Problem
  - **Step 1: Frequent Itemsets.**
  - **Step 2: Rule Generation.** For each frequent itemset ***L***, find all association rules that are above the minimum confidence ***c***.
    - For each subset ***Ls*** of ***L***, generate a rule ***Ls*** → ***(L-Ls)***
      - When ***L*** is a frequent itemset, the antecedent ***Ls*** is also frequent
      - **Option #1** enumerating **(naive)**: single pass to compute the confidence for all possible rules
        - For example, confidence(A,B → C,D) = support(A,B,C,D)/support(A,B)
      - **Option #2** generative **(recommended)**: If association rule A,B,C → D is below the minimum confidence, so is A,B →C,D
        - So we can generate rules with "longer" part of consequent from those with "shorter" part of consequent
        - Stop generating any longer consequent when the current rule is of confidence < ***c***

In-class Exercise (Association Rule Generation)

- **Generate Association Rules from the frequent itemset obtained in Practice**

TID	Transaction
T1	1, 3, 4, 6
T2	2, 3, 5, 7
T3	1, 2, 3, 5, 8
T4	2, 5, 9, 10
T5	1, 4

Given:
Minimum support ***s*** = 2
Minimum confidence ***c*** = 0.7
Frequent 3-itemsets are {2, 3, 5}

Try to generate all association rules from the given frequent 3-itemsets?

- Generate Association Rules in **Exercise**

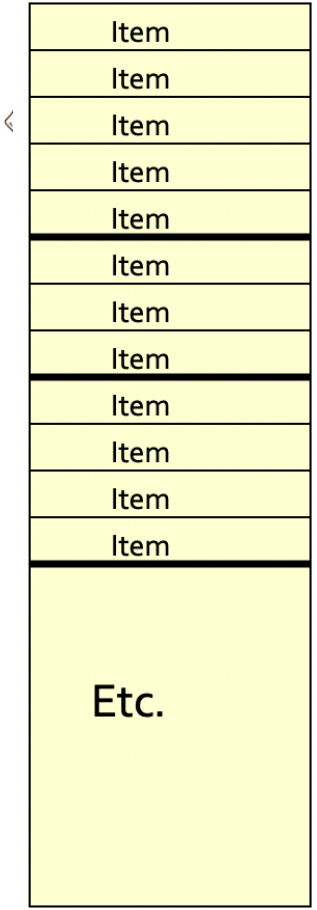
TID	Transaction
T1	1, 2 ,5
T2	2, 4
T3	2, 3
T4	1, 2, 4
T5	1, 3
T6	2, 3
T7	1, 3
T8	1, 2, 3, 5
T9	1, 2, 3

Given:
Minimum support ***s*** = 2
Minimum confidence ***c*** = 0.4
Frequent 3-itemsets {1,2,3} and {1,2,5}

Try to generate all association rules from the given frequent 3-itemsets?

Big Data Challenge Discussion: Computing to Find Frequent Itemsets

- Computation Model
  - Large-scale data is kept in flat files
    - Stored in disk
    - Stored basket by basket
    - Each basket may be small in size
    - But we have a large number of baskets and possible items
    - Computing process
      - Generate ***k***-itemsets (k=1,...,m) (what to count)
        - Nested loops to generate itemsets
      - Count the occurrence of each ***k***-itemset (whether frequent or not)
        - Reading baskets in order, to sum for the occurrence count



Items are positive integers, and boundaries between baskets are –1.

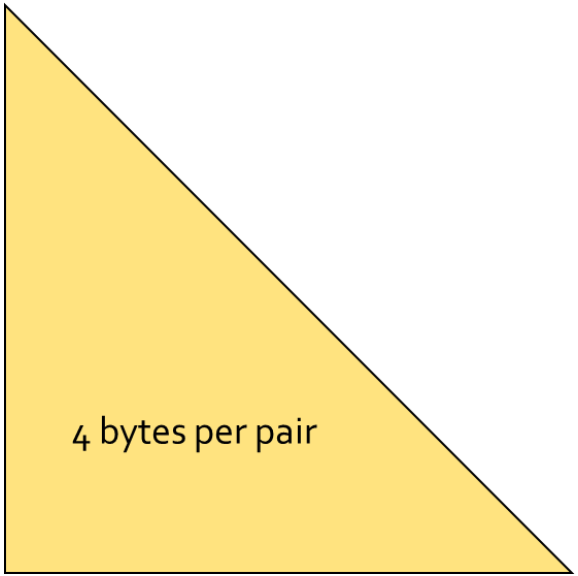
- Computation Cost
  - Essential cost of mining disk-resident data is the number of disk I/Os
  - Association-rule algorithms read the data in passes - baskets are read in order
  - The cost of algorithm can be measured by the number of passes
    - More passes means higher costs incurred using the algorithm to compute
  - Main memory is a critical resource
    - The number of different things we can count depends on memory
  - Number of passes may trade off with the memory that can be used by the algorithm

- Consider finding frequent 2-itemset (extend to larger sets later)
  - In practice, the most challenging part often turns out to be finding the frequent pair of items because frequent 2-itemsets are more common than frequent k-itemsets when k > 2:
    - The number of possible ***k***-itemsets grows with ***k*** (i.e., $\propto 2^k - 1$)
    - The probability of being frequent drops with size ***k*** (i.e., all subsets up to size k-1 should be frequent)
    - Taken the above two considerations together, growth in the former develops at a slower pace than the decline in the latter
  - Ideally
    - We would like to generate all the itemsets but only keep track of those end up being frequent

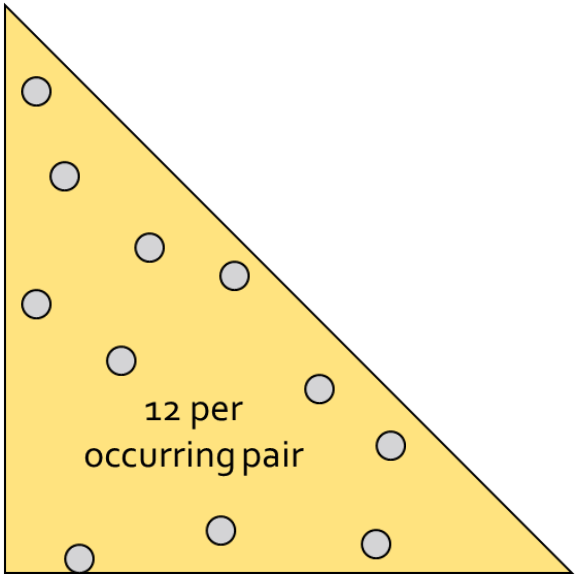
- Consider finding frequent 2-itemset (generalizable to larger sets)
  - **Naive Approach**
    - Read file once, counting the occurrences of every pair in the main memory
      - Suppose $|I| = n$ items that generate $n(n-1)/2$ pairs by two nested loops
    - Fails if it exceeds main memory (the number of items n goes up to 100K for Walmart (or 10B for webpage))
      - Given 10^5 items, counts are stored as 4-byte integers
      - Number of paris 10^5(10^5-1)/2 ≈ 5*10^9
      - As a result, it takes 4*5*10^9 bytes (20 GB) of main memory

- Consider finding frequent 2-itemset (generalizable to larger sets)
  - Naive Approach
    - Two alternative approaches to keep track of counts
      - Approach #1: Matrix - Build a matrix and store counts for all possible pairs in it.
        - Only requires 4 bytes for each pair (but many pairs!)
      - Approach #2: Triplets table - Maintain a table of triplets $[i, j, c]$ where the count of item index pair $\{i, j\}$ is c.
        - Suppose count integers and item IDs take 4 bytes each, it needs 12 bytes to store one triplet as long as its count > 0
        - In addition, some additional overhead for hashing (that quickly locate the focal pair to update in the table)

Comparing the Two Approaches



Triangular Matrix



Triples

Comparing the Two Approaches

- **Approach 1: Triangular Matrix**
  - **n** = total number items
  - Count pair of items {*i*, *j*} only if *i*<*j*
  - Keep pair counts in lexicographic order: {1,2}, {1,3},…, {1,*n*}, {2,3}, {2,4},…,{2,*n*}, {3,4},…
  - Total number of pairs $\frac{n(n-1)}{2}$;total bytes= $2n^2$ (**Triangular Matrix** requires 4 bytes per pair)
- **Approach 2** uses **12 *bytes*** per occurring pair *(but only for pairs with count > 0)*
  - Beats Approach 1 if less than **1/3** of possible pairs actually occur

Apriori Algorithm

- Apriori Approach Recap
  - One pass for each ***k-itemset***
  - Needs room in main memory to count each candidate ***k*-itemset**
  - For typical market-basket data and reasonable support (e.g., 1%), ***k*** =2 would require the most memory
- Possible extensions
  - Association rules with intervals
    - Men over 65 have 2 cars (treating intervals as items)
  - Association rules when items are in a taxonomy (hierarchy of items)
    - Bread, Butter → FruitJam ➔ BakedGoods, MilkProduct → PreservedGoods
  - Adapt (Lower) the support ***s*** as itemset gets bigger (adaptive threshold)